09

APIs de Alta Performance

Laravel na prática

Contratos enxutos, versionados e confiáveis para integrar sistemas.

Roteiro da aula

- Problema real no projeto base
- API Resource
- Otimização de payload
- Versionamento por URL
- Versionamento por Header
- Webhooks
- Webhooks robustos
- Recebimento de webhooks
- Projeto do módulo

Problema real no projeto base

Um parceiro precisa consultar horários disponíveis e receber atualizações sobre os agendamentos.

A API deve manter o mesmo contrato durante a evolução do sistema, responder com apenas os dados necessários e entregar eventos sem duplicar efeitos no sistema parceiro.

API Resource

Definição

API Resource transforma o model em um contrato JSON explícito.

Por que usar

Evita expor colunas internas e mantém datas, nomes e tipos consistentes para todos os clientes.

Exemplo de código

```
final class AppointmentResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'starts_at' => $this->starts_at->toISOString(),
            'status' => $this->status,
        ];
    }
}
```

Otimização de payload

Objetivo

- Reduzir tempo de resposta, tráfego e trabalho desnecessário no banco.
- Aplicação
- Selecione campos, pagine listagens e carregue somente relações solicitadas. Meça o resultado antes e depois da mudança.

Exemplo de código

```
Appointment::query()
    ->select(['id', 'starts_at', 'status'])
    ->when(
        $request->boolean('include_professional'),
        fn ($query) => $query->with('professional:id,name')
    )
    ->paginate(20);
```

Versionamento por URL

Uso

A versão faz parte da URL e permite manter contratos incompatíveis em paralelo.

Vantagem

É simples de descobrir, documentar e testar.

Cuidado

Extraia regras compartilhadas para não duplicar o domínio.

Exemplo de código

```
Route::prefix('v1')->group(function () {
    Route::apiResource(
        'appointments',
        AppointmentController::class
    );
});

GET /api/v1/appointments
```

Versionamento por Header

Uso

O cliente escolhe a versão pelo cabeçalho Accept, enquanto a URL permanece estável.

Vantagem

Evita URLs diferentes para o mesmo recurso.

Cuidado

Exige documentação e observabilidade mais cuidadosas.

Exemplo de código

```
GET /api/appointments
Accept: application/vnd.clinicflow.v1+json

$version = $request->header('Accept');

return match ($version) {
    'application/vnd.clinicflow.v1+json'
        => AppointmentV1Resource::collection($items),
    default => abort(406),
};
```

Webhooks

Definição

Webhook envia um evento HTTP para outro sistema após uma mudança relevante.

- Contrato
- Cada entrega precisa de identificador único, nome do evento, data de ocorrência e dados mínimos para o consumidor decidir o próximo passo.

Exemplo de código

```
{
  "event_id": "evt_01J...",
  "event": "appointment.created",
  "occurred_at": "2026-10-18T14:30:00Z",
  "data": {
    "appointment_id": 1842,
    "status": "confirmed"
  }
}
```

Webhooks robustos

Entrega confiável
- Envie em fila, aplique timeout curto e repita falhas temporárias com backoff.
- Segurança
- Assine o corpo com HMAC e registre tentativa, resposta e duração.
- Idempotência
- O event_id permite que o consumidor ignore entregas repetidas.

Exemplo de código

```
final class SendAppointmentWebhook implements ShouldQueue
{
    public int $tries = 5;

    public function backoff(): array
    {
        return [10, 30, 120, 300];
    }

    public function handle(WebhookClient $client): void
    {
        $client->send($this->event);
    }
}
```

Recebimento de webhooks

Fluxo de entrada

- Valide a assinatura sobre o corpo bruto antes de interpretar o JSON.
- Depois da validação
- Registre event_id com índice único, responda rapidamente e envie o processamento para a fila. Uma repetição deve retornar sucesso sem executar o efeito novamente.

Exemplo de código

```
$payload = $request->getContent();
$expected = hash_hmac('sha256', $payload, $secret);

abort_unless(
    hash_equals($expected, $request->header('X-Signature')),
    401
);

$event = WebhookEvent::firstOrCreate(
    ['event_id' => $request->string('event_id')],
    ['payload' => $request->json()->all()]
);

ProcessWebhook::dispatch($event);
return response()->noContent();
```

Projeto do módulo

Missão

Construir uma API de agenda consumida por um sistema parceiro e entregar eventos de agendamento por webhook.

Critérios de conclusão

- O endpoint usa Resource, paginação e versão explícita. O envio ocorre em fila, assina o payload e registra cada tentativa. O receptor valida a assinatura e impede processamento duplicado.

Checklist final

Ao final, o aluno deve conseguir
- definir contratos JSON com API Resources;
- reduzir payloads sem criar N+1;
- escolher uma estratégia de versionamento;
- enviar webhooks com assinatura, timeout e retry;
- receber eventos com validação e idempotência;
- medir o efeito das otimizações na API.